

Laboratorio di Architetture Software e Sicurezza Informatica [AAF1569]

8 - Behavior-Driven Development



SAPIENZA
UNIVERSITÀ DI ROMA

Part of therein contents are based on slides of Prof. Armando Fox

DIAG

Dipartimento di Ingegneria
informatica, automatica e gestionale
Antonio Ruberti

Software projects may fail

Why?

- they don't do what customers want
- they are late
- they are over budget
- they are hard to maintain and evolve
- all of the above

Different methodologies have been invented to attack these problems. The Agile methodology tries to do so for many common types of software.

Recall that Agile does not switch phases (and people) over time from development mode to maintenance mode. You're always maintaining...

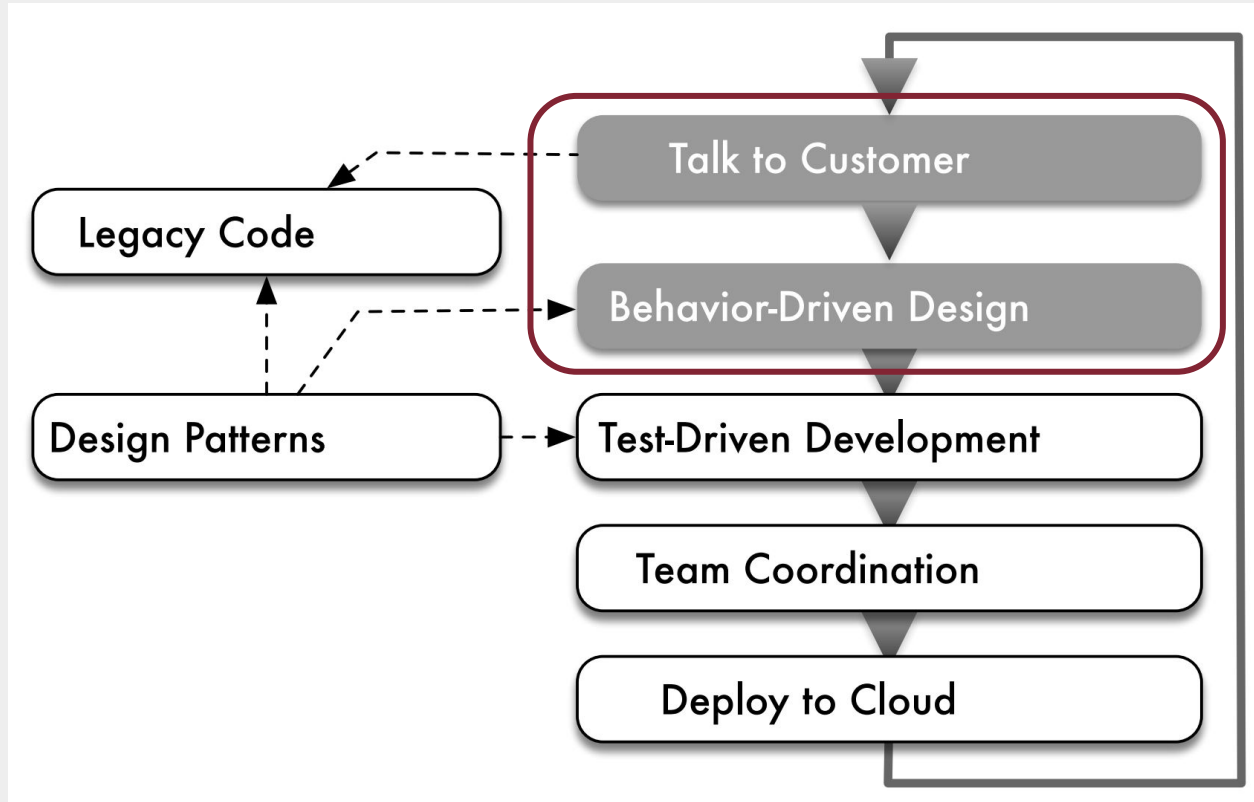
Agile methods

- *“Individuals and interactions over processes and tools”*
- *“Working software over comprehensive documentation”*
- *“Customer collaboration over contract negotiation”*
- *“Responding to change over following a plan”*

Some typical traits:

- _> **Embrace change**
- _> **Continuously refine** a working but incomplete prototype
- _> Rely on quick, constant **customer feedback**
- _> Emphasis on **test-driven** development
- _> **User stories** to reach agreement

An iteration of the Agile software lifecycle



Behaviour-Driven Development (BDD)

Behaviour-Driven Development (BDD) **asks questions** about the behavior of an application **before and during** the software **development process**

Validation (“build the right thing”), not just verification (“build the thing right”).

By asking questions about the app’s behavior, BDD is a way for software teams to work that **closes the gap** between business people and technical people:

- shared understanding of the problem to be solved
- working in rapid, small iterations to increase feedback
- produce **documentation automatically checked** against app’s behaviour

Behaviour-Driven Development (BDD)

User stories: concrete, real-world examples that illustrate how we want the system to behave. They replace the design documents from other approaches.

BDD concentrates on the behavior of an app instead of its implementation. It will be, for example, Test-Driven Development to take care of the latter.

User stories come from the Human Computer Interface (HCI) community, where they developed them using **3x5 cards** containing one to three sentences in everyday nontechnical language.

User stories must be testable, small enough to implement in one iteration, and have business value on their own.

BDD steps

BDD encourages working in rapid iterations, continuously **breaking** down your **user's problems into small pieces** that can flow through your development process as quickly as possible.

Three steps:

1. **[Discovery]** Take a small upcoming change to the system – a **User Story** – and talk about concrete examples of the new functionality to explore, discover and agree on the details of what's expected to be done.
2. **[Formulation]** Document those examples in a way that can be automated, and check for agreement.
3. **[Automation]** Implement the behaviour described by each documented example, starting with an automated test to guide code development

Discovery: User Stories

A User Story is small piece of valuable functionality used for planning and prioritising work on an agile team

1-3 sentences in informal language

Describes **how the application is expected to be used**, specifically it illustrates a **feature** of a software system. Written from the perspective of an end user.

The Connextra template is very popular:

*As a < **type of user** >,
So that < **I can achieve some goal** >,
I want to < **do some task** >*

A user story example with RottenPotatoen

Feature: Add a movie to RottenPotatoes

As a movie fan

So that I can share a movie with other movie fans

I want to add a movie to RottenPotatoes database

Scenario: Add a movie

Given I am on the RottenPotatoes home page

When I follow "Add new movie"

Then I should be on the Create New Movie page

When I fill in "Title" with "Hamilton"

And I select "PG-13" from "Rating"

And I select "July 4, 2020" as the "Released On" date

And I press "Save Changes"

Then I should be on the RottenPotatoes home page

And I should see "Hamilton"

SMART User Stories

What makes a user story good?

- Specific
- Measurable
- Achievable
- Relevant
- Timeboxed

SMART User Stories

What makes a user story good?

- **Specific**

“Feature: User can search for a movie (vague)”

Feature: User can search for a movie by title (specific)”

SMART User Stories

What makes a user story good?

- **Measurable**: each story should be testable. In practical terms, there are known expected results for some good inputs.

*“Feature: RottenPotatoes should have good response time
(unmeasurable)”*

*Feature: When adding a movie, 99% of Add Movie pages
should appear within 3 seconds (measurable)”*

SMART User Stories

What makes a user story good?

- **Achievable:** ideally, you must implement one story in one Agile iteration. If you are getting less than one story per iteration, you should subdivide these stories into smaller ones.
- **Relevant:** a user story should have business value to one or more stakeholders. To drill it down, keep asking them why it does.

The **minimum viable product** (MVP) is a subset of the full set of features that when completed has business value in the real world.

SMART User Stories

What makes a user story good?

- **Timeboxed:** stop developing a story once you've exceed the time budget. Either you give up (and divide it into smaller ones) or reschedule what is left according to a new estimate.

If dividing doesn't help, go back to the customers to find the **highest value part** of the story that you can do quickly.

Formulation: Cucumber

We seek for a user story format template that can be read by both humans (customers and developers) and computers.

They shouldn't look like code, so that they are clear to the customer and can be used to reach agreement, but they shouldn't be free-form either.

Cucumber is a tool that can turn customer-understandable user stories into both:

- **acceptance tests** (to ensure the customer is satisfied)
- and **integration tests** (to ensure the interfaces between modules have consistent assumptions and communicate correctly).

In the Cucumber context, a user story refers to a single **feature** with one or more **scenarios** that show different ways a feature is used.

Formulation: Gherkin with Cucumber

Gherkin is a set of grammar rules that makes plain text structured enough for Cucumber to understand. Acceptance criteria are Connextra template-alike.

Gherkin serves multiple purposes:

- **Unambiguous executable specification**
- **Automated testing** using Cucumber
- **Document how the system actually behaves**

Gherkin documents are stored in `.feature` text files

Gherkin Structure

<https://cucumber.io/docs/gherkin/reference/>

Keywords:

- **Feature**
- Rule
- Example (or **Scenario**)
- **Given, When, Then, And, But** for steps (or *)
- Background
- Scenario Outline (or Scenario Template)
- Examples (or Scenarios)

A set of separate files (*step definitions*) defines the Ruby code to test these steps

Feature

The purpose of the Feature keyword is to **provide a high-level description of a software feature**, and to group related scenarios.

You can add free-form text underneath Feature to add more description.

```
Feature: Guess the word
```

```
    The word guess game is a turn-based game for two players.
```

```
    The Maker makes a word for the Breaker to guess. The game  
    is over when the Breaker guesses the Maker's word.
```

Description for a Feature ends when you start a line with one of the keywords Rule, Example, or Scenario.

Example/Scenario

This is a concrete example that illustrates a business rule.

It consists of a **list of steps**. Examples follow this same pattern:

- Describe an **initial context** (Given steps)
- Describe an **event** (When steps)
- Describe an **expected outcome** (Then steps)

Scenarios often focus on “happy” paths for a feature, but it is also important to agree with the customer on what should happen when things go wrong.

Step: Given

Given steps are used to **describe the initial context** of the system, set up some preconditions.

The purpose of Given steps is to put the system in a known state before the user (or external system) starts interacting with the system (in the When steps).

Examples:

- Mickey and Minnie have started a game
- I am logged in
- Joe has a balance of £42

Step: When

When steps are used to **describe an event** or an action.

It is strongly recommended you only have a **single** When step per Scenario.

Examples:

- Guess a word
- Invite a friend
- Withdraw money

Capbara is a Ruby library written which simulates how an user interacts with your application. Cucumbers uses it to simulate actions in a web browser.

Step: Then

Then steps are used to **describe an expected outcome**

The step definition of a Then step should use an **assertion to compare the actual outcome** (what the system actually does) **to the expected outcome** (what the step says the system is supposed to do).

Examples:

- See that the guessed word was wrong
- Receive an invitation
- Card should be swallowed

Gherkin Example

Feature: Multiple site support

Only blog owners can post to a blog, except administrators,
who can post to all blogs.

Background:

Given a global administrator named "Greg"

And a blog named "Greg's anti-tax rants"

And a customer named "Dr. Bill"

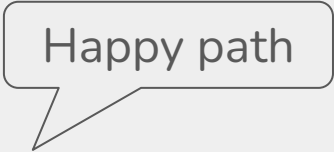
And a blog named "Expensive Therapy" owned by "Dr. Bill"

Scenario: Dr. Bill posts to his own blog

Given I am logged in as Dr. Bill

When I try to post to "Expensive Therapy"

Then I should see "Your article was published."



Happy path

Gherkin Example

Background:

Given a global administrator named "Greg"

And a blog named "Greg's anti-tax rants"

And a customer named "Dr. Bill"

And a blog named "Expensive Therapy" owned by "Dr. Bill"

Sad path

Scenario: Dr. Bill tries to post to somebody else's blog, and fails

Given I am logged in as Dr. Bill

When I try to post to "Greg's anti-tax rants"

Then I should see "Hey! That's not your blog!"

Scenario: Greg posts to a client's blog

Given I am logged in as Greg

When I try to post to "Expensive Therapy"

Then I should see "Your article was published."

Happy path

Step Definitions

Step definitions connect Gherkin steps to programming code

A step definition carries out the action that should be performed by the step, that is it hard-wires the specification to the implementation => Ruby code

Regular expressions (regexes) are used to match English phrases in steps of scenarios to step definitions

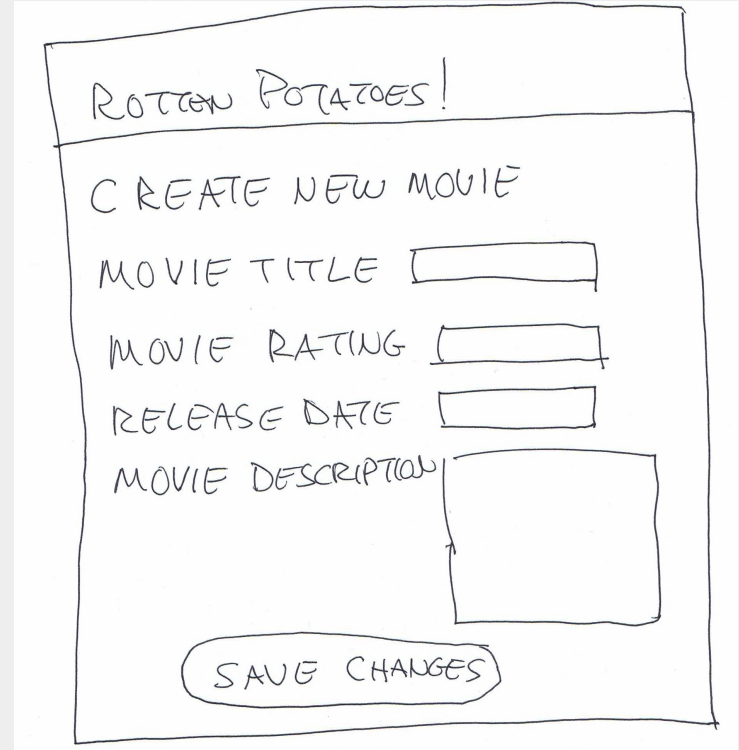
Lo-Fi User Interface Sketches and Storyboards

We usually **need to specify a user interface** (UI) when adding a new feature since many SaaS applications interact with end users.

Part of the BDD task is often to propose a UI to match the user stories

_> **lo-fi** (low-fidelity) **UI** and the paper prototypes **sketches**.

Ideally, you make sketches for all the user stories that involve a UI.



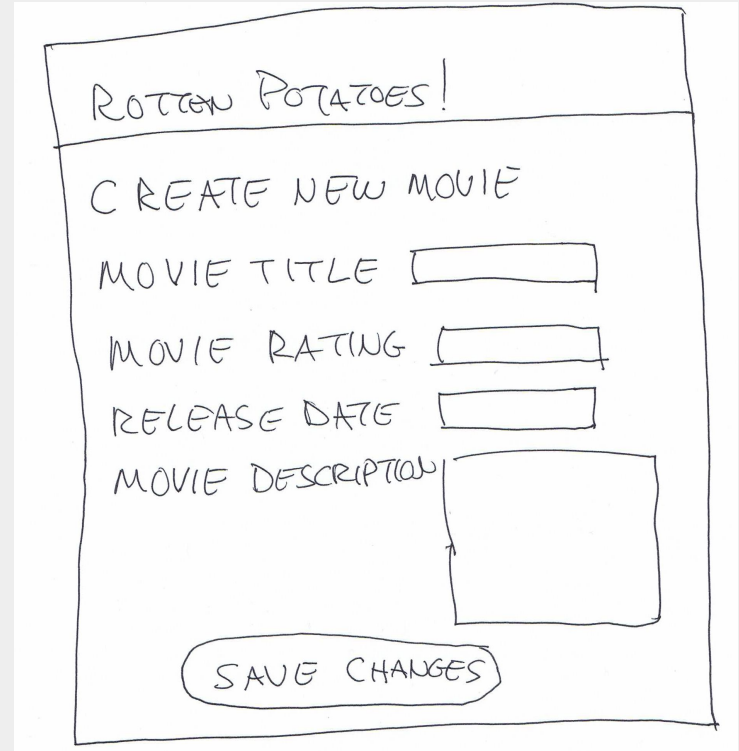
Lo-Fi User Interface Sketches and Storyboards

As many SaaS applications interact with users, we usually need to **specify a user interface** (UI) when adding a new feature.

Part of the BDD work is often to propose a UI to match the user stories. The HCI community proposes kindergarten tools to generate UI mockups:

_> **lo-fi (low-fidelity) UI** approach

These paper prototypes are called **sketches**. Ideally, you make sketches for all the user stories that involve a UI.

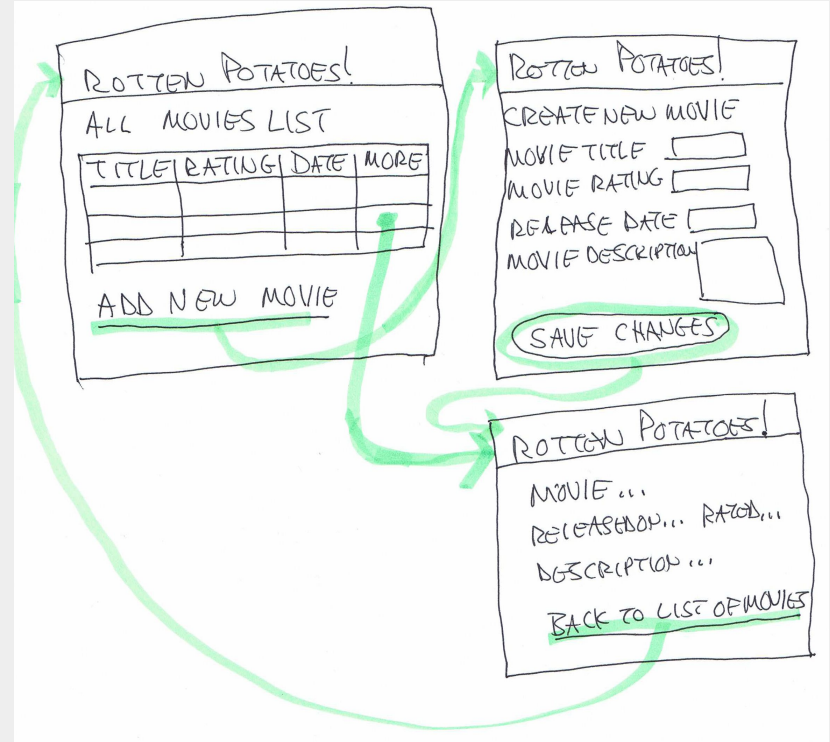


Lo-Fi User Interface Sketches and Storyboards

A lo-fi sketch shows what the UI looks like at one instant in time. But we also need to show **how the sketches work together as a user interacts with a page.**

Storyboarding: the storyboard for a UI is typically a tree or graph of screens driven by different user choices.

To make a storyboard, you must think about all the user interactions with a web app: pages or sections of pages, forms and buttons, and popups.



From Lo-Fi User Interface Sketches to HTML/CSS

After drawing the sketches and storyboards, you are ready to write HTML.

With lo-fi, the key is to first get a good **overall structure** from your sketches and do minimal CSS (if any at all) to get the view to look more or less like your sketch.

Start the process by looking at the lo-fi UI sketches and split them into blocks of the layout of your Rails app. Use HTML `<div>` tags for obvious layout sections.

There is no need to make it pretty until after you have everything working!

Adding CSS styling, images, and so on can happen after it work. You can resort to off-the-shelf CSS frameworks like Bootstrap as they facilitate producing an acceptable-looking prototype quickly.

Some takeaways from BDD

BDD is perfect for projects where the **requirements are poorly understood or rapidly changing**. User stories also make it easy to break projects into small increments, while the use of 3x5 cards and paper mockups keeps nontechnical customers involved in the design and prioritization of features.

In some scenarios, it may be difficult or too expensive to have continuous contact with the customer (for example, some they may not want to participate).

BDD may also not scale to very large software development projects or to safety critical applications. For both, plan-and-document is a better match.

Some takeaways from BDD

Another potential downside is that the project could satisfy customers but not result in a good software architecture, which is important for maintaining it. In these cases, identifying **design patterns** that match the circumstances and refactor the code when necessary may ameliorate the quality of the architecture.

BDD may not seem initially the natural way to develop software. But once you master it, it can repay the effort many times over in the long run for such apps.

BDD also meets the momentum in the Ruby community (which loves testable, good-looking and self-documenting code) in favor of test practices that specify behaviors both as a way to document developer intent for the app and to provide executable acceptance tests for it.